

Chapter 1: Introduction

- Interrupt Vector: Associating interrupt type to what kind of interrupt was raised.
- Trap / Exception: Software-driven Interrupt.
- Non-Maskable Interrupt Lines: Interrupts that can't be ignored.
- Maskable Interrupt Lines: Interrupts that can be ignored.
- Interrupt Service Routines / Interrupt Handlers: Routine that runs when an interrupt runs.
- DMA - Direct Memory Access: Bypassing memory and I/O can access memory.
- Asymmetric Multiprocessing: Not all CPUs are equal - Special CPUs for special purposes.
- Symmetric Multiprocessing: All CPUs are equal. All can perform same task.
- NUMA - Non-Uniform Memory Access System: Multiple memories with multiple CPUs
- Storage-Area Network (SAN): Clustered Systems running space.
- Clustered Systems: Multiprocessing but instead it is on system level.
- DLM (Distributed Lock Manager): Synchronization between systems for shared resources.
- Symmetric and Asymmetric Clustering: Master nodes with underneath workers: Asymmetric. All systems can work similar tasks.
- HPC (High Performance Computing): Powerful systems in Clusters.
- Bootstrap Program: Minimal Piece of Code to boot the system.
- System Daemons: Running system processes in the backend.
- Multiprogramming
 - Batch processing
 - Batches job scheduling
 - Wait till all jobs are done
 - Multitasking
 - Response time matters
 - Time slicing
- User / Privileged Mode
- Kernel / Supervisor Mode
- Dual Mode: User, Kernel Mode
- Multimode: User, Kernel, Guest Mode
- Timer: Interrupts from time to time
- Protection vs Security
 - Protection: Authorized access to the system
 - Security: Mitigating attacks like DDoS.

Chapter 2: Operating-System Structures

- Roles of OS

- User Interface
- Program Execution
- I/O Operations
- Communications
- File-system manipulation
- Communications
- Error Detection
- Resource Allocation
- Logging
- Protection and Security
- Command Line Interpreter - CLI
- System Calls: API provided by OS to run kernel instructions.
- Application Programming Interface (API)
 - Win32
 - POSIX
 - Java API (JVM)
- System-call interface
- System Call Parameter Passing
 - Registers
 - Heap
 - Table
- Types of System Calls
 - Process Control
 - File Management
 - Device Management
 - Information Maintenance
 - Communication
 - Protection
- Arduino
 - Single Task
 - Minimal Bootloader Code
- FreeBSD
 - Multitasking
- Applications as Operating System Specific
- Application Binary Interface (ABI): Low-level interface between binary files and operating systems
- Policy vs Mechanism

- Policy: Why we should do this?
- Mechanism: How can we do this?
- Monolithic Operating System
 - Linux
 - Tightly Coupled
- Layered Approach
 - Abstract Layers
 - More complex
- Microkernel
 - Minimal Kernel
 - Way more focus on user mode
 - Communication via Message Passing
- Mach
 - Basis of Darwin (MacOS X).
 - Mach Kernel
- Loadable Kernel Modules (LKMs)
 - Object Oriented.
 - When needed, modules are loaded
- Android
 - Open Source, extension of Linux in some way.
 - Uses Java API
- BIOS
- System Boot
- BPF Compiler Collection (BCC)
 - Tracing and debugging

Chapter 3: Process

- Heap
- Stack
- Data Section
- Text Section
- Process States
 - New
 - Ready
 - Running
 - Waiting
 - Terminated
- Process Control Block (PCB) / Task Control Block (TCB)

- Scheduling
 - Long-Term Scheduling (Batch OSes)
 - Medium-Term Scheduling
 - Short-Term Scheduling (Modern OSes)
- Context Switch
- Pid
- Process Creation
 - `fork()`
 - `exec()`
 - `wait()`
 - `CreateProcess()`
- Process Termination
 - `exit()`
- Orphan Processes: Still running even though their parents are exited. Taken up by root (systemd usually)
- Zombie Processes: That have exited but still have an entry in the kernel.
- Android Process Importance Hierarchy
 - Foreground first
- AOT vs JIT: Ahead of time vs Just in time
- Cooperating Processes: That share resources
- Independent Processes
- Interprocess Communication (IPC)
- Shared memory
- Message passing
- Producer-Consumer Problem
- Blocking vs Non-Blocking Transmission
- Direct vs Indirect Communication
- `mmap`
- `shm_open`
- Pipes
- Ordinary Pipes vs Named Pipes

Chapter 4: Threads & Concurrency

- Process vs Thread
- Parallelism vs Concurrency
- Data vs Task Parallelism
- Amdahl's Law
- User Threads vs Kernel Threads

- Multi-threading Models
- Pthreads
- `CreateThread()`
- Explicit Threading
- Implicit Threading
 - Thread Pools
 - Fork-Join Parallelism
 - OpenMP
 - Grand Central Dispatch (GCP)
 - Serial Queue
 - Concurrent Queue

- `pthread_testcancel()`
- Thread-local Storage (TLS)
- Lightweight Process (LWP)

Chapter 5: CPU Scheduling

- Preemption vs Non-preemption
- Dispatcher
- FCFS
- SJF
- Convoy Effect
- SRTF
- Exponential Averaging
- RR
- Process Starvation
- Aging
- Multilevel Queues (for priority)
- Multilevel Feedback Queue
- Thread Scheduling
- Contention Scope
- Process Contention Scope (User-Level Contention Scope)
- System Contention Scope (Kernel-Level Contention Scope)
- Hard affinity vs Soft Affinity
- CFS - Completely Fair Scheduler

Chapter 6: Synchronization Tools

- Race Conditions
- Entry, Critical, Exit, Remainder Sections
- Solution to Critical-Section Problem

- Mutual Exclusion
- Progress
- Bounded Waiting
- Peterson's Solution
- Memory Barrier
 - Strongly vs Weakly Ordered
- `test_and_set`
- `compare_and_swap`
- Monitors

Chapter 7: Synchronization Examples

- Dining Philosophers Problem
- Little's Law: Average number of items in a queue = Average arrival rate * Average time an item spends in a system.

Chapter 8: Deadlocks

- Resource Allocation Graph
- Circular Wait
- Request Edge
- Assignment Edge
- Safe State
- Claim Edge
- Banker's Algorithm